Chainlink FluxAggregator approach vs OCR

by Valya Zaitseva

5 March 2026

While reading about Chainlink's FluxAggregator and On-Chain Reporting (OCR), I wondered whether the DAI/USD price feed, when queried via `latestRoundData`, would exhibit FluxAggregator-like behaviour.

In the original FluxAggregator design, the `answer` may change several times during a round as additional oracle submissions arrive. This means that the `latestRoundData` for **the same** `roundId` must deviate till it stabilises because the price stabilisation lasts for some period of time:

- The new round **starts when the minimum number** of nodes have reported new data to the proxy - at this moment, the `latestRoundData` is calculated and returned to those who pull data from the new round.
- But the collection of node reports continues, which means that the `latestRoundData`, literally `answer` and `updatedAt` timestamp, are being changed for some period of time till the **maximum number of submissions is reached** - at this point, the `answer` is updated the last time and stays the same till the end of the round.

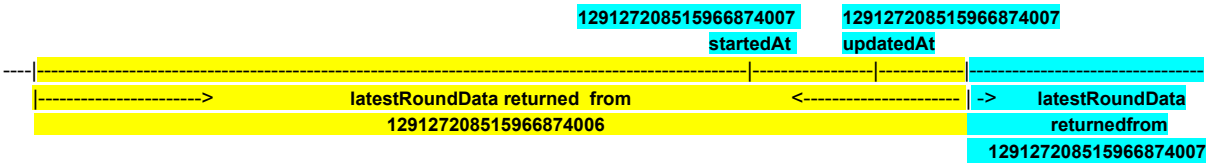
I observed the moment of round change and didn't catch that price dynamic because the FluxAggregator approach was replaced with OCR:

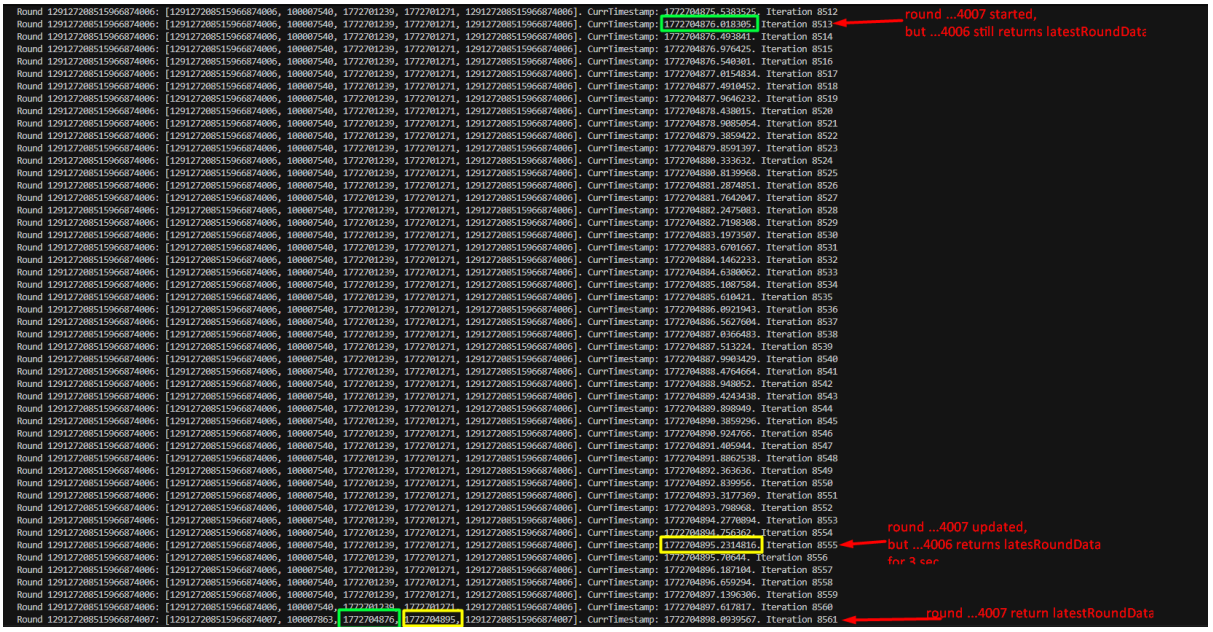
```
Round 12912720851596687400 5: [12912720851596687400 5, 100005772, 1772697547, 1772697623, 12912720851596687400 5]. Iteration 701
Round 12912720851596687400 5: [12912720851596687400 5, 100005772, 1772697547, 1772697623, 12912720851596687400 5]. Iteration 702
Round 12912720851596687400 5: [12912720851596687400 5, 100005772, 1772697547, 1772697623, 12912720851596687400 5]. Iteration 703
Round 12912720851596687400 5: [12912720851596687400 5, 100005772, 1772697547, 1772697623, 12912720851596687400 5]. Iteration 704
Round 12912720851596687400 5: [12912720851596687400 5, 100005772, 1772697547, 1772697623, 12912720851596687400 5]. Iteration 705
Round 12912720851596687400 5: [12912720851596687400 5, 100005772, 1772697547, 1772697623, 12912720851596687400 5]. Iteration 706
Round 12912720851596687400 5: [12912720851596687400 5, 100005772, 1772697547, 1772697623, 12912720851596687400 5]. Iteration 707
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 708
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 709
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 710
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 711
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 712
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 713
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 714
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 715
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 716
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 717
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 718
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 719
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 720
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 721
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 722
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 723
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 724
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 725
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 726
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 727
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 728
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 729
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 730
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 731
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 732
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 733
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 734
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 735
Round 12912720851596687400 6: [12912720851596687400 6, 100007540, 1772701239, 1772701271, 12912720851596687400 6]. Iteration 736
```

On the screenshot above, it is obvious that once the round has changed from 129127208515966874005 to 129127208515966874006, the `latestRoundData` returned the same `answer` all the way till the end of the round (`answer` = 100007540). This is not the behaviour of the FluxAggregator approach. This is OCR.

My further observation showed what actually happens when the round changes. In the screenshot below, it is clear that:

1. The round 129127208515966874007 started at 1772704876, but it returned `latestRoundData` 22 seconds later, at 1772704898.
2. The final `answer` at the aggregator state was updated at 1772704895, but it returned it 3 seconds later, at 1772704898





There is a significant gap between the round actual start (i.e. the price change) and the new `latestRoundData` to be returned. **This means that the price feed returns an outdated price for 22 seconds.** Does this delay matter for protocols relying on `latestRoundData` for liquidations?